

ECE391: Computer Systems Engineering Machine Problem 3

Fall 2025

Checkpoint 1: Fri Oct 31 18:00 CDT
Checkpoint 2: Fri Nov 21 18:00 CDT
Checkpoint 3: Fri Dec 5 18:00 CDT

Illinux 391

Contents

1	Introduction	4
2	Using the Group Repository	4
3	The Pieces	5
3.1	Getting started	5
3.2	Work Plan	5
3.3	Requirements	6
4	Testing	7
5	What to Hand in	7
5.1	Checkpoint 1: Filesystem and Drivers and Program Loading, (Oh My!)	7
5.1.1	Group Repository	7
5.1.2	VIRTIO Block Device	7
5.1.3	Read-Only Filesystem	8
5.1.4	Rambunctious RAM Disk	9
5.1.5	Locked and	10
5.1.6	...(ELF) Loaded	10
5.1.7	Cache Me If You Can	10
5.1.8	Existing Files	11
5.1.9	File Modifications	12
5.1.10	Running Your Code	12

5.1.11	Troubleshooting/Debugging	12
5.1.12	Checkpoint 1 Handin	12
5.2	Checkpoint 2: Virtual Memory and Process Abstraction	12
5.2.1	Given Code	12
5.2.2	Vicious Virtual Memory	13
5.2.3	Pesky Processes	14
5.2.4	Suspicious Syscalls	17
5.2.5	Pedantic Path Parsing	18
5.2.6	Fully Implemented Filesystem	18
5.2.7	Additional Modifications	19
5.2.8	Checkpoint 2 Handin	20
5.3	Checkpoint 3: Forks, Preemptive Multitasking, Pipes, and Shell	21
5.3.1	Friendly Forking	21
5.3.2	Perniciously Meticulous Preemptive Multitasking	21
5.3.3	Listing at Last!	22
5.3.4	Pervasive Pipes	22
5.3.5	Welcome to (S)hell	22
5.3.6	Universal Unix-like Utils	24
5.4	Checkpoint 3 Modifications	26
6	Grading	27
7	Appendix A: The File System	28
7.1	File System Overview	28
7.2	File Abstractions	30
7.3	Building the File System	31
7.4	The Blob	32
8	Appendix B: Uniform I/O	33
8.1	Overview	33
8.2	Uniform I/O Operations	33
8.3	Opening Uniform I/O objects	33

8.4	Listing Objects	34
8.5	Uniform I/O Diagram	35
9	Appendix C: The Process Abstraction	36
9.1	History	36
9.2	Overview	36
9.3	Context Switching between User-Mode and Supervisor-Mode	36
10	Appendix D: RISC-V Paging	39
10.1	Illinix Mappings	39
10.2	Sv39	40
11	Appendix E: Troubleshooting	41
11.1	Debugging with gdb	41
11.1.1	Debugging the Kernel	41
11.1.2	Debugging User Programs	42
11.1.3	Useful gdb Commands	42
12	Appendix F: Games	44
12.1	Overview	44
12.2	Trek	44
12.3	Zork	44
12.4	Rogue	44
12.5	Game Quirks	45

1 Introduction

Read the whole document before you begin, or you may miss points on some requirements.

In this machine problem, you collaborate to develop the core of an operating system roughly based on Unix Version 6, with modern concepts peppered in where appropriate. You'll implement interrupt logic, user threading (à la MP2), kernel and application paging, initialize some devices and a filesystem with the VIRTIO interface, and create a system call interface to support various system calls. The operating system will support running several tasks ("threads") spawned by a number of user programs; programs will interface with the kernel via system calls.

Don't worry, these aren't all "boring" programs—you'll get to run some cool games, too.

The goal for the assignment is to provide you with hands-on experience in developing the software used to interface between devices and applications, *i.e.*, operating systems. You should notice that the work here builds on concepts from the other machine problems. Many of the abstractions used here (*e.g.*, the VIRTIO interface) have been simplified to reduce the effort necessary to complete the project, but we hope that you will leave the class with the skills necessary to extend the implementation that you develop here along whatever direction you choose, by incrementally improving various aspects of your system.

2 Using the Group Repository

You should receive access to a shared repository with your group members. Your group has a two-digit group number; your group number should be released in a Piazza post, and your repository should be named something like `mp3_group_xx`, where `xx` is your number.

It is required to run the following commands on each computer that you clone the repository to, in order to make sure the line endings are set to LF (Unix style):

```
git config --global core.autocrlf input
git config --global core.eol lf
```

Another reminder, you will need to setup your git config with your name and email as stated in the group contract. Please review the contract form for instructions on how to do this. This is a requirement, failure to set this up **will result in a penalty**.

Some other tips:

As you work on MP3 with your teammates, you may find it useful to create additional branches to avoid conflicts while editing source files. Remember to `git pull` each time you sit down to work on the project and `git commit` and `git push` when you are done. Doing so will ensure that all members are working on the most current version of the sources. It is highly likely that you will benefit from proper usage of the `git stash` command, to correctly retain desired (local) changes when doing a pull.

When using Git, keep in mind that it is, in general, bad practice to commit broken sources. You should make sure that your sources compile correctly before committing them to the repository. Make sure not to commit compiled `*.o` files, besides what we have given you. You can modify your `.gitignore` in any way you

want.

Finally, merge your changes into the `main` branch by each checkpoint deadline, as **this is the only branch we will use for grading**.

3 The Pieces

The basic OS is provided, but when you first get it, it won't properly do a lot of the things we expect an OS to do: perhaps most importantly [†], it can't play any cool games!

For simplicity, we will stick to text-mode graphics (for the most part), but your OS will, by the end, run the games from previous MPs as well as some new ones. We've included a few helpful pieces that will allow you to debug easier, such as `kprintf`. We **highly** encourage use of `gdb` to debug. Print statements can only take you so far. See **Appendix E** for details.

3.1 Getting started

In order to effectively work on this MP, you will need to develop knowledge of a lot of different and difficult concepts. The documentation on the course website and lectures will give you background information, but the best way to learn is to read and understand the code we have provided. This includes the `Makefiles`, `.ld` linker files, and `.c/.h` source files.

This MP is difficult, which is why we do not expect you to work alone. While you cannot share code or discuss details with other groups or anyone outside your group, you should work together with your team to get unstuck and build a common understanding.

3.2 Work Plan

“Work expands so as to fill the time available for its completion.” - Cyril Parkinson

This project is somewhat daunting, and will require efforts from all your team members. You should partition the work accordingly to allow independent progress by all team members.

Setting up a clean testing interface will also help substantially with partitioning the work, since group members can't finish and test components before your groupmates finish the other parts (yet). The abstractions suggested should allow for some spots where a “working part” can be substituted with a functionally equivalent placeholder, so to speak—more on that later.

While splitting up the work allows for you to make more progress, it is still crucial that you spend time working together to integrate all parts. You should also be maintaining active communication between group members to make sure you all have an understanding of how all your code works. Even if you did not work on a specific section, we expect you to be familiar with how the code works and be able to explain what it and how it fits into your kernel.

Throughout the first part of this semester and in most/all of your previous classes, MPs were more structured. You were given a set of functions to write, and you only modified those functions. One of the goals of this

[†]Some may disagree that this is the most important thing, but who wants no games?

class is to make you into a more confident and thoughtful programmer by having you practice software design. We have deliberately left the implementation of certain sections open-ended. It is up to you and your group to find a way to meet the requirements - as with the real world, there is no “perfect” solution. The only requirement that we impose is that you follow the function interfaces that we have already specified - feel free create your own helper functions or creative implementations.

3.3 Requirements

In order to be compliant with our autograder, we will enforce the following requirements:

1. You may not add any new files to the repo other than the files from your MP2 that we specify
2. Any helper functions you write must be declared as `static`
3. You may not modify any provided function headers

Additionally, you should only modify the following files for CP1:

- `cache.c`
- `cache.h`
- `elf.c`
- `ktfs.c`
- `ktfs.h`
- `main.c`
- `dev/ramdisk.c`
- `dev/ramdisk.h`
- `dev/vioblk.c`

4 Testing

For this project, we strongly recommend that you write and run unit tests with adequate coverage of your source code.

As your operating system components are dependent on one another, you may find it useful to unit test each component individually to isolate any design or coding bugs.

We have provided ample scaffolding to help you create unit tests, and we strongly recommend developing comprehensive test suites to verify the functionality at all stages of this MP. A sample `test_main.c` and `testsuite_1.c` are provided for housing your unit tests, along with a corresponding target in the `Makefile`. You should modify these files as needed to thoroughly test your kernel.

As you add more components to your operating system, we encourage you to add corresponding tests that verify the functionality of each component at the interface level.

Keep in mind that passing all of your unit tests does not guarantee bug free code. However, the test suite provides a convenient means to run your tests frequently without having to re-debug older components as you add new functionality.

5 What to Hand in

5.1 Checkpoint 1: Filesystem and Drivers and Program Loading, (Oh My!)

The primary motivation of this checkpoint is to get 2 test programs, which we've given you, to run. `hello` will print some text to the UART screen, then stop. `trek` will run the same text-based Star Trek game that you know and love.

Rather than do this in a “hacky” way, we want to set up some key infrastructure now which will pay dividends as the project continues on.

Note: Please ensure that all the functions that we specify do not induce a kernel panic on an invalid input. You should return an error code (which one is up to you) if possible, otherwise do nothing.

For the checkpoint, you must have the following accomplished:

5.1.1 Group Repository

You must have your code in the shared group repository, and each group member should be able to demonstrate that they can read and change the source code in the repository.

5.1.2 VIRTIO Block Device

An operating system in general must communicate with external devices. One such device is obviously the real drive/disk (virtual, in this case) which contains programs and other files you want your operating system to have access to.

In order to set up this device (and any others down the line), we will need to set up the necessary framework for the VIRTIO block device. Your group must finish the implementation based on the VIRTIO documentation linked on the course website.

You may find it especially helpful to read sections 2.1-2.7, 3.1, 4.2, and 5.2 and recall your implementation of `viornng.c`. Skip anything related to “legacy” interface.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

1. `void vioblk_attach(volatile struct virtio_mmio_regs * regs, int irqno)`
2. `int vioblk_storage_open(struct storage * sto)`
3. `void vioblk_storage_close(struct storage * sto)`
4. `long vioblk_storage_fetch(struct storage * sto, unsigned long long pos, void * buf, unsigned long bytecnt)`
5. `long vioblk_storage_store(struct storage * sto, unsigned long long pos, const void * buf, unsigned long bytecnt)`
6. `vioblk_storage_cntl(struct storage * sto, int op, void * arg)`
7. `vioblk_isr(int irqno, void * aux)`

Also check the doxygen and the provided code for more information on the storage device struct. It is a little different than you previously implemented serial devices.

5.1.3 Read-Only Filesystem

Broadly speaking, your filesystem driver should provide a comfortable interface to open, read, and scan through files. In later checkpoints, you will add additional functionality.

Your `ktfs.c` file will need to interact with its backing device with some intermediate cache in `cache.c` to actually interact with the “physical” (well, virtual) device, so be sure that you understand what’s going on in files related to both the cache and the backing device.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

These functions should be written in `ktfs.c`:

1. `int mount_ktfs(char * name, struct * cache)`
2. `int ktfs_open(struct filesystem * fs, const char * name, struct uio ** uioptr)`
3. `void ktfs_close(struct uio * uio)`
4. `long ktfs_fetch(struct uio* uio, void * buf, unsigned long len)`

5. `int ktfs_cntl(struct uio * uio, int cmd, void * arg)`
6. `void ktfs_flush(struct filesystem * fs)`

Note the use of a new `uio` struct here. This is how your kernel will interact with “file-like” objects (such as files!). See **Appendix B** for more information on the `uio` struct.

In order to use the filesystem, we have provided a `mkfs_ktfs` function (see **Appendix A**) that generates a filesystem image for you. Be sure your implementation works for very large image sizes. This filesystem image is mounted by QEMU as a drive (using the `Makefile` we provide) and is accessible through `VIRTIO`.

For CP1, you will need to implement “`fcntl`”: `FCNTL_GETEND`, `FCNTL_GETPOS`, and `FCNTL_SETPOS` (Note: we are not requiring `FCNTL_SETEND` for this checkpoint since currently the FS is read only). Each opened file will need to keep track of its `pos`, and calling `read` with `bufsz` bytes will return `[pos, pos + bufsz - 1]` bytes as well as advance `pos` to `pos + bufsz`.

(Hint: implement the `ramdisk` device and its related functions to test your KTFS filesystem driver as well as the `cache` independently from `vioblk`.)

See **Appendix A** for additional details.

5.1.4 Rambunctious RAM Disk

You may wonder why we need a `ramdisk` device if we already have a `vioblk` device? The `ramdisk` device is a helper backing device that allows you to test your filesystem and ELF loader without using `vioblk`. Within your linker script `kernel.ld`, there is a section from `_kimg_blob_start` to `_kimg_blob_end` that you can use to place your whole filesystem image or an ELF file to load (see **Appendix A** about the blob).

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

For better unit testing and debugging, you must implement the `ramdisk` device “driver” in the `dev` directory. The following is a list of functions you need to implement:

1. `void ramdisk_attach()`
2. `int ramdisk_open(struct storage * sto)`
3. `void ramdisk_close(struct storage * sto)`
4. `int ramdisk_cntl(struct storage * sto, int cmd, void * arg)`
5. `long ramdisk_fetch(struct storage * sto, unsigned long long pos, void * buf, long bytecnt)`

If you would like to also test your ELF loader with the `ramdisk` device, you will need to interface it with a `uio`. To do this, see **Appendix B** for details on how `uio` and the filesystem are connected.

5.1.5 Locked and ...

Locks prevent concurrency issues when multiple threads are accessing the same resource. We have already provided locking objects and functions for you to utilize as needed. However, it is on you to determine where locks will be necessary for your OS in order to prevent concurrency issues.

Hint: In order to prevent deadlocks, an exiting thread must release all held locks.

More information about the functions we have given can be found on the Doxygen site linked on the course website.

5.1.6 ... (ELF) Loaded

One of the key roles of the operating system is to be able to run other programs.

We want to be able to run many user-level programs, but for now you can focus on `hello` and `trek`. While we've given you the pre-compiled binary of `trek`, you'll have to compile `hello` using the `usr/Makefile`. Because of this, you also have access to `hello.c`. All the binaries are in a format called ELF (Executable and Linkable Format), which has a specific layout — it is the standard for Unix and Unix-like systems, historically, which means it is still very relevant. See the **Tools, References, and Links** page on the course website for the Linux manual page on ELF. Your loader will only need to deal with the program headers, not sections, so focus on that documentation.

Notice that since `elf_load` should support any compliant UIO interface, that we can in general load an ELF from “any source” as long as the proper functions are implemented in the given `uio_intf` (see **Appendix B**).

5.1.7 Cache Me If You Can

“Software gets slower faster than hardware gets faster.” -Wirth’s Law

This semester, you will implement a caching system to cache blocks from a backing interface. As you've learned in this course, communicating with devices is extremely slow relative to the CPU's clock cycle. Previously, you've implemented asynchronous communication (condition variables) so that while one thread is waiting on a device response, another thread can run.

While this significantly reduces the problem of “wasting” CPU cycles, it does not eliminate the problem of latency - the original thread still must wait a long time for the device's response. A cache is a commonly used way to reduce this latency.

Once you complete this checkpoint, your backing interface will be the VIRTIO block device, but we will refer to it generically as the backing interface in this document. Rather than reading/writing a block directly from/to the backing interface, we first check whether the block exists in the cache. If the block exists in the cache, we access the block via the cache rather than sending a new request to the backing interface. If the block does not exist in the cache, we read it from the backing device into the cache. Note that you may or may not need to evict a block currently in the cache in order to bring in this new block.

Your cache may have any level of associativity and may be write-back, write-through, or some other concoction. Please note that we are intentionally leaving the specific details of the cache vague; this is intended

to be a design exercise.

Some additional scenarios/specifications for the cache are as follows (these would be good test cases for you to write):

- Initially (when the OS is first started), the cache is empty.
- The total number of blocks that the cache can hold must be 64.
- If the cache is initially empty and we read 64 contiguous blocks into it (e.g. 5, 6, 7, ..., 68), all 64 of those blocks must remain in the cache. This means that a read within any of those blocks should not generate a request to the backing device.
- If `cache_get_block()` is called and block k is read into the cache, block k should remain in the cache at least until `cache_get_block()` is called again (note that block k may remain in the cache for longer). This is to say, if we call `cache_get_block()` once and do not call it again, the block that was cached must remain in the cache.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

You will implement the following 4 functions in `cache.c`:

1. `int create_cache(struct storage * sto, struct cache ** cptr)`
2. `int cache_get_block(struct cache * cache, unsigned long long pos, void ** pptr)`
3. `void cache_release_block(struct cache * cache, void * pblk, int dirty)`
4. `int cache_flush(struct cache * cache)`

As a reminder, you can also create any other helper functions that you need for your cache implementation.

5.1.8 Existing Files

During MP2, you worked with the PLIC, UART, and some other devices. You will be re-using that code for this checkpoint. You should add the following files into `sys` from MP2. They must be fully functional and (besides `thread.c`) should have the same functionality as a completed MP2. You can collaborate with your MP3 groupmates to choose whose MP2 code to use.

- `sys/dev/rtc.c/.h`
- `sys/dev/uart.c/.h`
- `sys/dev/viorng.c`
- `sys/plic.c/.h`
- `sys/timer.c/.h`
- `sys/thread.c/.h`
- `sys/thrasm.s`

5.1.9 File Modifications

For this MP there are a few small changes to make to the given code:

- Replace all inclusions of `assert.h` to `misc.h`
- Add the line `#include see.h` to `thread.c`
- Add the line `#include <stddef.h>` to `thread.c`

5.1.10 Running Your Code

Finally, once you have all of your code completed, you will need to finish `sys/main.c` to run your program. We've left some comments on how to run `trek`, you may also find it helpful to refer to your MP2 CP3 `main` function implementation. You can also run `hello` or another user program that you create.

5.1.11 Troubleshooting/Debugging

See **Appendix E** for more information about debugging and common issues.

5.1.12 Checkpoint 1 Handin

For handin, your work must be completed and pushed to the `main` branch of your team's GitHub remote repository by the deadline.

For this checkpoint you must complete the following:

- Read-write operations on `vioblk`
- Read-only filesystem
- Read operations on `ramdisk`
- Read-write operations on the cache
- ELF Load

5.2 Checkpoint 2: Virtual Memory and Process Abstraction

5.2.1 Given Code

Linked on the course website are some extra files which are needed for the Checkpoint 2 implementation. Add them to your repo before starting Checkpoint 2 development. If you are starting CP2 before CP1 deadline be sure not to include any of these changes on your `main` branch, since some of these change will affect your CP1 behavior.

Files to add to the `sys` directory:

1. `memory.c`
2. `memory.h`
3. `process.c`
4. `process.h`
5. `syscall.c`
6. `heap0.c` - This should replace your existing `heap0.c` and uses your virtual memory functions.

Additionally, we have provided more games that use U-mode and system calls, including a new version of `trek`. You should place these files in your `usr/bin` directory. See **Appendix F** for more details on games.

5.2.2 Vicious Virtual Memory

In operating systems, paging is a memory management technique that allows non-contiguous allocation and efficient use of storage by dividing a process's address space into fixed-size units called pages. Page tables exist to maintain mappings between physical and virtual memory. When a process accesses a virtual address, the page table is queried to find the corresponding physical address. A page fault occurs when a process tries to access a virtual address that is not mapped to a physical address. If you handle a page fault that occurs within the User-owned virtual memory address space (`USER_START_VMA` to `USER_END_VMA`) you should allocate a new page and map that address as U-mode accessible. This is known as "lazy allocation" or "demand paging". For page faults that occur in other virtual memory regions, you should terminate the user program.

The operating system needs to support both single-page and multi-page allocations. When a process requires memory, the system selects the best-fit chunk from the free page list (the smallest chunk that fits the requested number of pages) to ensure optimal memory utilization. Pages can also be freed individually or in groups to ensure efficient reuse. When memory is no longer needed, in order to prevent security risks and memory leakage, the system must reset. This ensures clearing of non-global pages and freeing the associated memory.

Flags are used in memory management to control access permissions for maintaining security and track page usage. They are critical in allocating memory, mapping and unmapping of pages, validating memory access, as well as handling page faults. Access faults occur when a process tries to access a virtual memory address that they do not have permissions to access.

Note: This checkpoint will only contain a single memory space. You will not need to create new memory spaces for this checkpoint besides the "main" memory space.

We've given you a file, `memory.c`, where you will implement all the functions declared in `memory.h`. Keep in mind, many of these functions have overlap and it may be useful to look at the provided helper functions as well as write some of your own. You must write the following functions for this checkpoint:

1. `void memory_init(void)`, most of this is given besides the initializing the free chunk list
2. `void reset_active_mspace(void)`
3. `mtag_t discard_active_mspace(void)`

4. `void * alloc_phys_page(void)`
5. `void * alloc_phys_pages(unsigned int cnt)`
6. `void free_phys_page(void * pp)`
7. `void free_phys_pages(void * pp, unsigned int cnt)`
8. `unsigned long free_phys_page_count(void)`
9. `void * map_page(uintptr_t vma, void * pp, int rwxug_flags)`
10. `void * map_range(uintptr_t vma, size_t size, void * pp, int rwxug_flags)`
11. `void * alloc_and_map_range(uintptr_t vma, size_t size, int rwxug_flags)`
12. `void set_range_flags(const void * vp, size_t size, int rwxug_flags)`
13. `void unmap_and_free_range(void * vp, size_t size)`
14. `int validate_vptr(const void * vp, size_t len, int rwxug_flags)`
15. `int validate_vstr(const char * vs, int rug_flags)`
16. `int handle_umode_page_fault(struct trap_frame *tfr, uintptr_t vma)`

Some of these functions may build off of others. Some functions will also be called by functions used to implement processes and syscalls (§5.2.3).

In order to set up virtual to physical page mappings in your kernel, you must have a way to keep track of what physical pages are available to be mapped. To do this, we have created the “`free_chunk_list`”. A `page_chunk` is a contiguous region of physical memory addresses. Each `page_chunk` contains a pointer to the next `page_chunk` in the list as well as a size (in pages). Initially, all of the memory from the end of the heap (`heap_end`, which is page-aligned) until the end of RAM (defined in `conf.h`) are free physical pages. You must modify `memory_init` to place all of these pages (in a single chunk) on the `free_chunk_list`.

Allocating and freeing physical pages must also interact with the `free_chunk_list`. To allocate physical page(s), you should go through the `free_chunk_list` and find a chunk that is greater than or equal the number of contiguous physical pages that you need to allocate. If there is no chunk large enough, you can panic. If there is a chunk, you should break off an appropriately-sized piece and provide a pointer to the start of the physical address range that was allocated. To free physical page(s), you can simply place the chunk back on the `free_chunk_list` - no need to coalesce chunks together.

Consult **Appendix D** for more information about virtual memory.

5.2.3 Pesky Processes

The process abstraction is one of the key abstractions of an operating system. A process can be defined informally as just a “running user program”. A user often wants to run multiple processes at once which requires common resources like processing power, devices, and memory to be managed.

An instance of a process structure contains everything that a process owns and uses internally. Each user process is actually just a wrapper around a kernel thread. What this means is whenever a user process is created, a process struct will have to be initialized to contain the information below:

1. An identifier for the current process
2. An identifier for the kernel thread related to this process
3. An identifier for the memory space of the process
4. A list of UIO interfaces for this process ; remember that a UIO interface can currently represent
 - A terminal device
 - An open file
 - A block device
 - An in-memory buffer

In this checkpoint, all user processes will share the same memory space, dubbed the “main” memory space. The execution lifecycle of a process will be as follows

1. The kernel launches in S-mode.
2. `procmgr_init` is called, creating a process struct around the main thread
3. `process_exec` jumps to user mode and starts executing a user program. Effectively turning the main kernel thread into a user process
4. When the user process exits, the kernel exits

Our kernel space process API is made up of the functions below. These functions will reside in `process.c` and should be written by you unless if stated otherwise:

1. `void procmgr_init(void)` (Provided)
 Initializes processes globally by initializing a process structure for the main user process (`init`). The `init` process should always be assigned process ID (PID) 0.
2. `int process_exec(struct uio * exeio, int argc, char ** argv)`
 Executes a program referred to by the UIO interface passed in as an argument. We only require a maximum of 16 concurrent processes.
 Executing a loaded program with `process_exec` has 4 main requirements:
 - (a) First any virtual memory mappings belonging to other user processes should be unmapped.
 - (b) Next the executable should be loaded from the UIO interface provided as an argument into the mapped pages. (Note: After an executable is loaded into memory space, the file should be closed.)

- (c) Finally, the thread associated with the process needs to be started in user-mode. (Hint: An assembly function in `trap.s` would be useful here)

Context switching was relatively trivial when both contexts were at the same privilege level (i.e. machine-mode to machine-mode switching or supervisor-mode to supervisor-mode switching), but now we need to switch from a more privileged mode (supervisor-mode) to less privileged mode (user-mode).

Doing so requires using clever tricks with supervisor-mode CSRs and supervisor-mode instructions. Here are some tips to consider while implementing a context switch from supervisor-mode to user-mode

- i. Consider instructions that can transition between lower-privilege modes and higher privilege modes. Can you repurpose them for context switching purposes?
- ii. If you did repurpose them for context switching purposes, what CSRs would you need to edit so that the transition would start the thread's start function in user-mode?

It's a useful exercise to try to figure out how such an approach could work with the CSRs and supervisor-mode instructions on your own. However, implementation on its own is a sufficient challenge and we don't require you to figure this out. You can read **Appendix C** to find out how you can carry out a context switch between user-mode to supervisor mode.

3. `void process_exit(void)`

Cleans up after a finished process by reclaiming the resources of the process. Anything that was associated with the process at initial execution should be released. This covers:

- Process memory space
- Open UIO interfaces
- Associated kernel thread

You will also have to modify your current threading library to accommodate for processes. To do this you will have to both declare the specified functions in your `thread.h` file as well as define them in your `thread.c` file.

The following functions will have to be implemented by you:

1. `struct process * thread_process(int tid)`
2. `struct process * running_thread_process(void)`
3. `void thread_set_process(int tid, struct process * proc)`

You also will have to add a `struct process * proc` to your thread struct

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

5.2.4 Suspicious Syscalls

You will need to implement a series of system calls (syscalls) for this checkpoint. The user program uses these to request actions from the kernel.

The syscalls you must implement for this checkpoint are:

1. `static int sysexit(void)`
2. `static int sysprint(const char *msg)`
3. `static int sysopen(int fd, const char *name)`
4. `static int sysclose(int fd)`
5. `static long sysread(int fd, void *buf, size_t bufsz)`
6. `static long syswrite(int fd, const void *buf, size_t len)`
7. `static int sysfcntl(int fd, int cmd, void *arg)`
8. `static int sysexec(int fd, int argc, char ** argv)`
9. `static int syswait(int tid)`
10. `static int sysusleep(unsigned long us)`
11. `static int sysfscreate(const char * name)`
12. `static int sysfsdelete(const char * name )`
13. `static int sysuiodup(int oldfd, int newfd)`
14. `void handle_syscall(struct trap_frame *tfr)`
15. `int64_t syscall(const struct trap_frame *tfr)`

The function `handle_syscall` is used for system call linkage, through it calling the `syscall` function. Looking into the `syscall.S`, we see that system call exceptions are generating using the `ecall` instruction. The exception should be handled by `smode_trap_entry_from_umode` in `trap.s`. This assembly code should call the `handle_umode_exception` function in `excp.c`. User-mode exception handling is used for implementing both system calls and page fault handling. You will need to write both of these functions.

`smode_trap_entry_from_umode` will be the entry point to the kernel when a trap is generated in U-mode. Similar to its S-mode equivalent, you must save registers to the trap frame, go to either the exception handler or interrupt handler, restore registers, and return. However, there are subtle differences that you will need to reason about and understand to make this function work. We highly recommend that you thoroughly read and understand `smode_trap_entry_from_smode` before attempting this function.

`handle_umode_exception` will be similar to `handle_smode_exception`, but keep in mind that we “gracefully” handle 2 exception types: page faults and environment calls. If one of these occurs (and is handled without

an error) you should return from this function through `smod_trap_entry_from_umode`, otherwise, you should exit the current process.

System calls pass as parameters pointers in user memory. This is a vulnerability since the kernel could read or write from invalid memory. Use the virtual memory validation functions to protect the kernel from malicious user programs.

5.2.5 Pedantic Path Parsing

You'll notice that the `_open` syscall does not take in the same mountpoint and filename that `open_file` does. Users will instead provide a path as the argument, using a `'/'` to differentiate between the mountpoint and the file. This provides greater abstraction from the filesystem and allows a more unified interaction between mountpoints (think nested directories). For example, if a user wishes to open the device `uart1`, then they will call `_open(1, "dev/uart1")` in order to specify the `dev` mountpoint.

This requires you to parse the input yourself. To do this, you must modify `fileys.c` to implement the following function

```
• int parse_path(char *path, char **mpnameptr, char **fnameptr)
```

See the doxygen for more details.

5.2.6 Fully Implemented Filesystem

In Checkpoint 1, you already implemented a read-only KTFS driver. In this checkpoint, you will extend your filesystem driver to allow for writing to files as well as file creation and deletion. Keep in mind that all previous read behavior should be maintained, and after this checkpoint, your filesystem should implement CRUD functionality with respect to files.

Writes to the disk should persist within the filesystem image even after QEMU shutdown. That is, if you run a program that creates a file and writes to it (assuming it is written to the `vioblk` device through the cache), then shut down QEMU, you should be able to open and read from that file the next time QEMU is launched. This also means that you can look at your filesystem image file after QEMU exits to help debug writes. Be careful that the writes are truly persisting to disk - if you have a write-back cache, they may only be written to the cache before QEMU exits. You must also support writes past the end of the file, such that if a user requests to write data at a position larger than the current end, the file should be expanded to accommodate this request. Remember to update both the current position AND the end position of the file.

Note: Writing to files is a bit trickier than reading from them, but there is a lot of overlap code. Try your best to follow DRY (Don't Repeat Yourself).

For this checkpoint, you should implement the following functions in `ktfs.c`:

1. `long ktfs_store(struct uio* uio, const void * buf, unsigned long len)`
2. `int ktfs_create(struct filesystem* fs, const char* name)`
3. `int ktfs_delete(struct filesystem* fs, const char* name)`

You must also implement `FCNTL_SETEND` in `ktfs_cntl`. `FCNTL_SETEND` should extend the length of the file and add additional data blocks if needed.

The dentries used in your filesystem must be contiguous at all times. That means, that there are no “holes” in the data blocks of dentries, even after `ktfs_delete` is called on a filename in the middle. You should also make sure to add `ktfs_create` and `ktfs_delete` functions to your filesystem interface.

It is very important that you write thorough tests for the filesystem. There are many possible edge and corner cases and we highly encourage you to create targeted tests to ensure that you handle all of them properly.

5.2.7 Additional Modifications

Due to the new virtual memory, process abstraction, and system calls, we are now able to run programs in User-mode instead of Supervisor-mode. This means that some of the code you wrote in Checkpoint 1 will no longer work. Here is a (non-exhaustive) list of modifications you’ll have to make to your existing code:

1. `elf.c` must be modified to use virtual memory. User programs should now be loaded between the `0x0C0000000UL` and `0x100000000UL` virtual addresses. If you used the provided `USER_START_VMA` and `USER_END_VMA` constants in `conf.h`, you should update them to these values as well. You will need to allocate and map the appropriate amount of pages, as well as set the appropriate flags in the page table when mapping the program segments.
2. `Makefile` needs to be modified to compile the new files provided. Add `memory.o`, `process.o`, and `syscall.o` to your `OBJS`.
3. `main.c` should be modified to initialize memory and the process manager. You should also remove `heap_init` from `main.c`, as `memory_init` will do that for you.
4. `trap.h` must be modified. You should change the `trap_frame_jump` function declaration from

```
extern void __attribute__((noreturn)) trap_frame_jump(struct trap_frame * tfr) to
extern void trap_frame_jump(struct trap_frame * tfr, void* sscratch) __attribute__((noreturn))
```
5. `usr/Makefile` needs to be modified to support the new User-mode program address space. We have already provided a linker file (`umode.ld`), so you only need to change the `UMODE` flag in `usr/Makefile` to 1. **Note: Once you make this change, you must recompile any user programs.**

Besides the modifications above, you should only need to modify the following files for CP2:

- `memory.c`
- `process.c`
- `syscall.c`
- `ktfs.c`
- `main.c`
- `elf.c`

- `excp.c`

As well as any fixes that you may or may not need to make to your CP1 files.

5.2.8 Checkpoint 2 Handin

For handin, your work must be completed and pushed to the `main` branch of your team's GitHub remote repository by the deadline.

For this checkpoint you must complete the following:

- Reading, writing, creation, and deletion of files within KTFS
- Correct virtual memory mappings
- Correct virtual memory functionality
- Process abstraction
- System calls from U-mode

We have given you new games in **Appendix F**, which uses *some but not all* of this functionality. While being able to run this program is a good sign, it does not necessarily mean that all functions are correct and you are handling edgecases properly. You should write your own testcases to verify functionality.

5.3 Checkpoint 3: Forks, Preemptive Multitasking, Pipes, and Shell

5.3.1 Friendly Forking

“Fork this, I’m out” -KIP

We **heavily** recommend to finish this first as `shell` is dependent on forking.

In Checkpoint 3 we need to support “forking” processes so that we can have multiple processes running in our system. We’re starting off with the `fork` syscall which has the following function signature.

- `static int sysfork(const struct trap_frame * tfr)`

To implement the fork system call you will have to fill out the following functions in `process.c` which are described in the doxygen.

- `int process_fork(const struct trap_frame * tfr)`
- `void fork_func(struct condition * done, struct trap_frame * tfr)`

You will also have to implement the following function in `memory.c` to use in `process_fork`. It is described in the doxygen.

- `mtag_t clone_active_mspace(void)`

Another thing to note is you will now have to deal with multiple memory spaces. This means that you have to switch between them when switching between processes. This change will have to be done in `thread.c`. You should also change the `kmalloc` in `create_thread` when allocating a stack page to a `alloc_phys_page` so that each new process gets a full page for their kernel stack. You will also have to change the corresponding `kfree` in `running_thread_suspend` to a `free_phys_page`.

You may be delighted to hear that the `fork` implementation here is virtually identical to how it’s done in “real” UNIX-like operating systems, yay!

5.3.2 Perniciously Meticulous Preemptive Multitasking

In this checkpoint you will implement preemptive multitasking for programs running in user mode. Preemptive multitasking is very similar to the cooperative multitasking from MP2 but instead of calling `thread_yield` synchronously through the program itself during execution, it is called asynchronously via the interrupt handler. This will preempt the currently running thread by causing it to suspend and hand over control to the next waiting thread. Remember, we will only preempt the thread if we are running programs in the user mode. This can be done in `handle_umode_interrupt`. This will allow switching between kernel threads (and therefore user processes) when forking creates additional ones (besides `main`).

However, this alone is not sufficient to implement preemptive multitasking, as user processes may hog the CPU and never yield control back to the kernel. To solve this, you will need to set up a periodic alarm that will trigger an interrupt at a regular interval, forcing the kernel to regain control and switch between threads. You will need to add this special alarm in the timer ISR in `timer.c`.

5.3.3 Listing at Last!

To finally round out your KTFS implementation, you will be adding support for listing files in your filesystem. Specifically, you will need to support opening a listing object when the filename is `NULL` or an empty string. This listing object should allow users to read the filenames of all files in the filesystem one by one.

To do this, you will need to modify your `ktfs_open` to return a listing `uioptr` when the user requests it. This will require implementing the following functions in `ktfs.c`:

- `int ktfs_listing_read(struct uio * uio, void * buf, unsigned long bufsz)`
- `int ktfs_listing_close(struct uio * uio)`

Take a look at the doxygen for more details on these specific functions, and **Appendix A** for more information on listing objects in general. You may also find it helpful to look at the `devfs` listing implementation in `device.c` or the `filesys` listing object in `filesys.c` as examples.

5.3.4 Pervasive Pipes

In this checkpoint, you will be adding support for UNIX-style one-way pipes in your operating system. You will create a Uniform I/O interface provider that would function similarly to your existing Uniform I/O interfaces and will include the logic required for sending data between different processes.

You will need to create two different Uniform I/O interfaces for this purpose, one for the writer-end and one for the reader-end of the pipe. Both will reference the same buffer, allowing the writer process to send data to the reader process. Further details can be found on the doxygen.

You will also have to define the following in `uio.h` and implement it in `uio.c`, further detail can be found on the doxygen

- `extern void create_pipe(struct uio ** wptr, struct uio ** rptr)`

You will have to write the full pipe implementation in `uio.c` in order to make your pipes implementation work. You may have to create static helper functions as well as functions to populate your Uniform I/O interface.

Finally, you will need to update your syscalls to support user process piping. To do this, you will need to implement an additional syscall `syspipe` associated with syscall number 20. Its function signature is as follows and more details can be found on the doxygen.

- `int syspipe(int * wfdptr, int * rfdptr)`

5.3.5 Welcome to (S)hell

At this point, you have implemented everything you need to have a fully functional Operating System! We will now shift our focus to the user side of your operating system, implementing programs that will run on top

of your OS. Specifically, you will be implementing a shell program that will allow users to interact with your kernel and run other user programs. We have provided the start of a shell program in `usr/progs/shell.c` that you will be responsible for implementing. You may modify this file in any way you see fit to implement the required functionality.

At a minimum, your shell must support the following features:

- Forking/Exec'ing user input programs
- Parsing program arguments and passing into `exec`, with the proper `argc` and `argv`
- Setup `STDOUT` and `STDIN` in the proper descriptors for each user program
- Input/Output redirection
- Piping between user programs

The specific implementation of shell is intentionally left vague. As long as your shell meets the requirements laid out above, specifics of implementation are left as a design exercise. Below are some example shell inputs and descriptions for each, note that the leading `#` is not part of the input.

```
# trek
```

This should fork and have the child run `_exec(trek_fd, 1, {"c/trek", NULL})` while the parent waits for the child to complete. Unlike in Checkpoint 2, you should return to your shell upon exiting the game (assuming your `fork` works as expected). Note that the command `trek` should be automatically mapped to the file `c/trek` in your filesystem. You should do this by first, checking if the command is a full path (contains a `/`), and if not, prepending `c/` to the command before trying to open it in your filesystem. You may find the helper functions provided in `string.h` useful for this, specifically `snprintf`.

You should also be able to specify the programs arguments, where subsequent values will be passed into the executable's `argv`.

```
# rogue c/savedata
```

This command should run as `_exec(rogue_fd, 2, {"c/rogue", "c/savedata", NULL})`. Note that the shell should ignore any leading, trailing, or repeat spaces when parsing the input. That is to say, the following commands should all be equivalent to the one above:

```
#   rogue c/savedata
```

```
# rogue      c/savedata
```

```
#      rogue  c/savedata
```

In general, user programs will perform all reads from `STDIN` (file descriptor 0) and all writes to `STDOUT` (file descriptor 1). Additionally, user programs may write to `CONSOLE_OUT` (file descriptor 2) to output directly to the console, which you may find useful for writing error messages from your shell. Your shell should set up these file descriptors appropriately for each user program it runs. By default, your shell should set all of these to be your console. However, users may redirect input and output to files as described below.

```
# wc < c/file
```

Is the same as first running `_fsopen(STDIN, "c/file")` followed by `_exec(wc_fd, 1, {"c/wc", NULL})`. In the case where `c/file` does not exist, your shell should print an appropriate error message to `CONSOLE.OUT` and should not execute `wc`. However, if `c/file` does exist, `wc` will then read its input from `file` instead of from the console. Similarly, for output redirection, `file` should be opened in `STDOUT`.

```
# echo hello world > c/file
```

If `file` is not in your filesystem, however, it should be created with `_fscreate` in this case, before `_exec(echo_fd, 3, {"c/echo", "hello", "world", NULL})` is called. Note that redirection should overwrite the contents of the file if it already exists.

The final feature this shell should support is piping. You can set up two programs to communicate with each other, setting a pipe to be one program's `STDIN` and another program's `STDOUT`.

```
# echo c/test1 c/test2 c/test3 | xargs rm
```

This command should pipe to get the file descriptors, create two processes to run concurrently, and finally execute both programs on each process (Hint: you will have to call `_fork` multiple times!). The first process' `STDOUT` should be set to the write-end of the pipe before calling `_exec(echo_fd, 4, {"c/echo", "c/test1", "c/test2", "c/test3", NULL})`, and the second process' `STDIN` should be set to the read-end of the pipe before `_exec(xargs_fd, 2, {"c/xargs", "rm", NULL})`.

You are more than welcome to extend your shell to include additional features as long as all specified feature requirements are met.

Once your shell is implemented, you should add it to your `usr/Makefile` under `ALL_TARGETS`, such that it compiles to the binary file `shell` in `usr/bin/`. You can then add this binary to your filesystem image by adding it to `ktfs.raw`. You should also modify your `main.c` to run your shell on startup.

5.3.6 Universal Unix-like Utils

Now that you have a working shell, you will need some additional user programs to run on it! From here on out, you may assume that any user program you write will be executed from your shell and should have the following function signature for its `main` function:

```
void main(int argc, char* argv[])
```

Where `argc` is the number of command line arguments (including the program name itself) and `argv` is a `NULL` terminating array of strings representing each argument.

You will be implementing a few simplified Unix-like utilities to run on your shell. These programs should be placed in `usr/progs/` under the name of the utility (for example, `usr/progs/date.c` for the `date` utility). These will be the **ONLY** files used in grading this section, so all of your code should be contained in these files. Do not modify any files outside of `usr/progs/` since they will not be used in grading. Each program should also be added to your `usr/Makefile` under `ALL_TARGETS`, so that they compile to binaries in `usr/bin/`. You should also add each binary to your filesystem image (`ktfs.raw`) such that they can be run from your shell.

The required utilities are as follows:

- `date` - A program that prints the current date and time to `STDOUT`. You will need to use the `rtc` device,

which will return the time in nanoseconds since January 1st, 1970 GMT. You will need to convert this value into a human-readable date and time format (for example, 05 Dec 2025 18:00:00). For simplicity, you may ignore leap seconds and output the time in GMT. However, you should account for leap years.

- `echo` - A program that outputs its command line arguments to `STDOUT`, separated by spaces, followed by a newline. Since you are not required to implement any special shell features (like quotes or escape characters), you can assume that each argument is separated by a single space.
- `cat` - A program that reads from the first command line argument file and writes its contents to `STDOUT`.
- `ls` - A program that lists all files in the specified “directory” (“`c`” for `ktfs`, “`dev`” for devices, or “`/`” for root) to `STDOUT`, one per line.
- `wc` - A program that reads from the first command line argument file and outputs the number of lines, words, and bytes in the file to `STDOUT`, separated by tabs. If no file is specified, it should read from `STDIN`.
- `touch` - A program that creates empty files with the specified names in the filesystem.
- `rm` - A program that deletes the specified files from the filesystem.
- `xargs` - A program that reads items from `STDIN`, separated by spaces or newlines, and executes the command specified in its arguments with those items as additional arguments.

For all utilities that take in a file as an argument, you may assume that an absolute path is passed in. For example, `cat c/file` should attempt to read from `file` in `ktfs`, while `cat file` can simply return an error.

You may implement additional utilities if you wish, but these are the minimum required. You should ensure that these programs can be run both individually and in combination with each other using pipes and redirection in your shell. Additionally, any error messages that occur should be printed to `CONSOLE.OUT` (file descriptor 2).

If you have any questions about these utilities, please first refer to the program’s actual man pages for expected behavior. Your formatting should match as closely as possible, but you are not required to implement any additional features besides those specified above (for example, you do not need to implement any flags or options for these utilities). You may also find it helpful to run these utilities on a real Unix-like system and replicate their behavior in your own implementations.

Note that as we delve further into userspace, we trust the code we execute less and less. Users may be malicious, write buggy code, or pass in invalid arguments that could crash your program. Therefore, your kernel should be robust against such misbehavior, returning appropriate error codes or even terminating the process if necessary. Your OS should never crash and only panic if the ability to continue execution is compromised (for example, out of memory conditions in the kernel). It is on you to ensure that this is the case. We highly recommend looking back through all of the code you implemented throughout this MP to find kernel errors that may be unexpectedly invoked by user code. We will be testing your OS with a variety of user programs, some of which may be intentionally malicious. In order to receive full credit, your OS must handle these situations gracefully and remain stable. By the end of this checkpoint, you should have a robust OS capable of running multiple user programs concurrently!

5.4 Checkpoint 3 Modifications

For this checkpoint, you should be able to show that all the core functionality required here works. Namely:

- Be able to run your shell, your `main.c` should run shell by default
- Be able support forking and pipes
- Be able to run multiple processes concurrently
- Be able to execute all specified user programs
- Be able to compile all the files associated with your kernel and user programs.

Additionally, you should only need to modify the following files:

- `process.c`
- `memory.c`
- `syscall.c`
- `intr.c`
- `uio.c/h`
- `ktfs.c`
- `timer.c`
- `usr/progs/shell.c` and other user programs in `usr/progs/`
- `main.c`

As well as any fixes that you may or may not need to make to your previous checkpoint files. Good luck and happy coding!

6 Grading

Your final MP score is a baseline score that will be adjusted for teamwork and for individual effort. Note that the “correct” behavior of certain routines includes interfaces that allow one to prevent buffer overflows (that is, the interfaces do not leave the size of a buffer as an unknown) and other such behavior. While the TAs will probably not have time to sift through all of your code in detail, they will read parts of it and look at your overall design. The rough breakdown of how points will be distributed will be periodically released on the website prior to checkpoint deadlines.

A Note on Teamwork

Teamwork is an important part of this class and will be used in the grading of this MP. We expect that you will work to operate effectively as a team, leveraging each member’s strengths and making sure that everyone understands how the system operates to the extent that they can explain it. In the final demo, for example, we will ask each of the team members questions and expect that they will be able to answer at a reasonable level **without** referring to another team member. **Failure to operate as a team will significantly reduce your overall grade for this MP.**

At the end of the MP there will be a teammate evaluation which will factor into your final MP grade.

You are also expected to adhere to the MP3 contract.

7 Appendix A: The File System

7.1 File System Overview

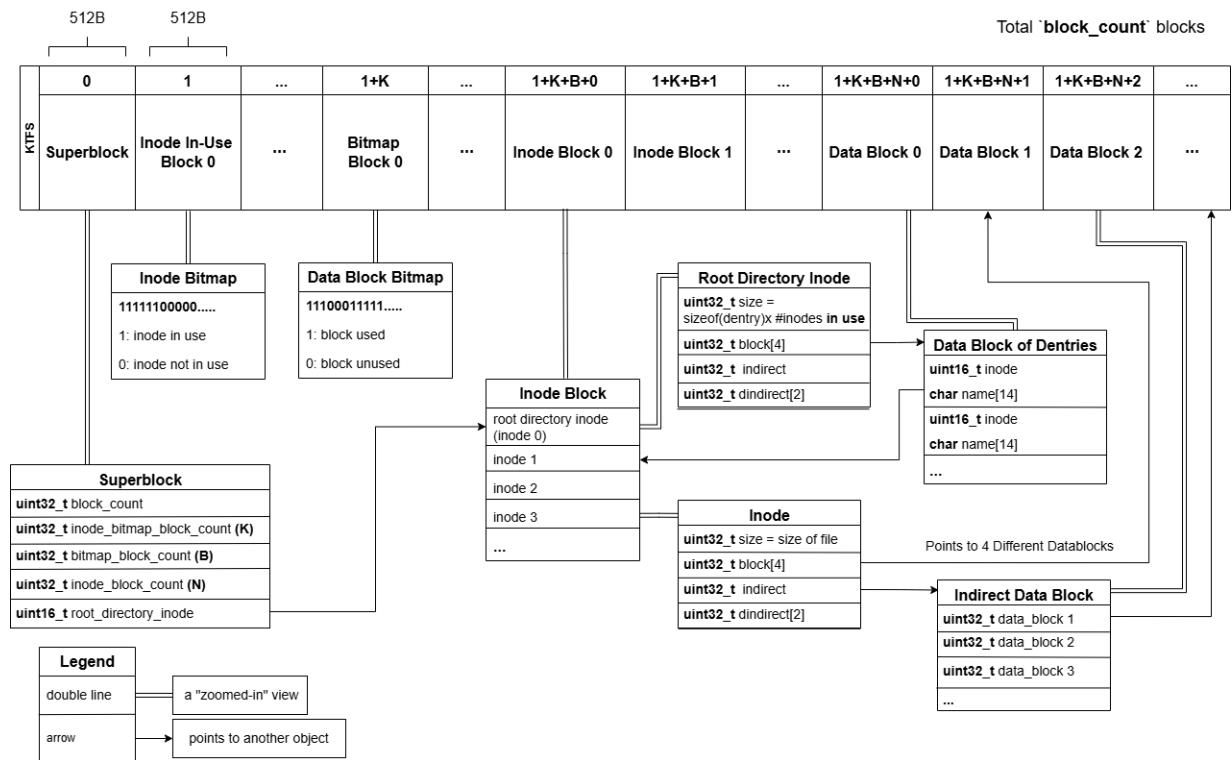
A filesystem can be considered an abstraction that allows for easy access of one or more “files” within a large chunk of data. For this MP, you will be implementing KTFS - a custom filesystem inspired by ext2.

First, we will provide definitions of important terms.

The *filesystem image* refers to a “blob” of data that contains everything needed to mount the filesystem and all the data associated with it.

The *backing device* for a filesystem refers to where the filesystem image is stored. In an actual computer, it’s typically stored on the hard drive. However, for this MP, the backing device will be the VIRTIO block device, which emulates a real hard drive. To test your filesystem without the VIRTIO block device, you can use a buffer in memory (see ramdisk).

The figure below shows the structure and contents of the filesystem image.



KTFS is a block-based filesystem. We use a block size of 512B, which coincidentally matches with the block size of the VIRTIO device. There are 5 types of blocks:

1. Superblock (stores basic statistics of the file system image)
2. Inode Bitmap blocks (keeps track of which inodes are in use)
3. Bitmap blocks (keeps track of which blocks are in-use)
4. Inode blocks (stores index nodes (inodes), which contain info about each file)
5. Data blocks (stores actual data)

The first block is called the **superblock**. The superblock has five fields:

- `uint32_t block_count` - the total number of block in the file system
- `uint32_t inode_bitmap_block_count` - the total number of inode bitmap blocks
- `uint32_t bitmap_block_count` - the number of bitmap blocks
- `uint32_t inode_block_count` - the number of inode blocks
- `uint16_t root_directory_inode` - the index of the root directory inode (see below)

These fields only take up the first 18B of the 512B, so the remaining bytes are unused. The fields in the superblock are set when creating the filesystem image (see §**Building the File System**) and should not be modified by your OS.

The **inode bitmap blocks** contain “bits”, as the name suggests. Each bit represents whether an inode entry is in-use. This is different from an inode block being in-use. For example, the 0th bit in the 0th inode bitmap block would correspond to the first 16B inode entry in Inode Block 0. This bitmap will need to be updated when removing or adding files (CP2).

The **bitmap blocks** make up another bitmap, where each bit indicates whether a block is used: 1 means used, 0 means free. For example, the 0th bit of the 0th bitmap block corresponds to the 0th block of the filesystem (i.e. the superblock). If a block is free, then it can be used later when writing to a file or creating a new file. When the filesystem image is created, the superblock, bitmap blocks, and inode blocks are all marked as in-use. During filesystem image creation, the “disk size” is set and fixed, which means the number of bitmap blocks is also fixed.

The **inode blocks** contain the index nodes (inodes) of the files in the filesystem. Each file is described by an inode that specifies the file’s size in bytes and the numbers of the data block (e.g. data blocks 1,5,10 ...) that make up the file. During filesystem image creation, the number of inode blocks is set and cannot change.

Each inode only provide direct access to 4 data blocks through an array within the inode. However, inodes additionally store 1 indirect data block number and 2 doubly-indirect data block numbers. An indirect data block number points to a data block that contains an array of data block numbers, instead of actual data. For example, if you want to access the 4th data block of a file, you will have to get the indirect data block number first (e.g. `inode.indirect = 11`), get data block 11 from backing device, and read the first `uint32_t` (first 4B) of data block 11 (e.g. 14). Then, go to data block 14 and there will be actual data. The idea is the same for doubly-indirect blocks, except there’s one more layer of indirection: the doubly-indirect data block number points to a data block that stores an array of indirect data block numbers, which further points to indirect data blocks that store an array of “real” data block numbers.

Each indirect/doubly-indirect data block contains 512B (just like any other data block), but only the part of the indirect/doubly-indirect data block that points to data blocks necessary to contain the specified size need be valid, so be careful not to read and make use of block numbers that lie beyond those necessary to contain the file data. The data blocks that make up a file are not necessarily contiguous or in any specific order. You must use the data block numbers in the order specified in the inode and the indirect/doubly-indirect data blocks to access the correct data in the correct order.

There's one special inode called the root directory inode. Instead of containing the actual data of a file in its specified data blocks, these data blocks contain the directory entries. KTFS does not support nested directories, so the root directory is the only directory that you are required to support. Each directory dentry (or dentries for short) contains an inode number and a string that contains the file name. The size of each dentry is 16B: 2B for the inode number and 14B for the file name. Note that because we need 1B for the null terminator, the maximum length of a file name is 13B (13 characters). The "file size" specified in the root directory inode is equal to (the number of dentries) $\times$ (the size of each dentry). Therefore, you can determine the number of files in the filesystem by checking the size field of the root directory inode. When you create or delete a file, you must add/remove the dentry for that file and update the size appropriately. For this file system, there is a 1:1:1 correspondence between files, dentries, and inodes. No two dentries should contain the same inode number or file name.

Note: Dentries **must** be contiguous to be able to use the file size properly. This is a strict requirement that filesystem images will follow and your filesystem driver must maintain. Keep in mind that "contiguous" here does not actually mean contiguous in memory, just that if you were reading the root directory inode like a file, the first `size` bytes would be the dentries.

In most cases, the **data blocks** can be seen as normal chunks of data in a file. That means when you open a file, you should be reading data from and writing data to these actual data blocks. However, because of the root directory inode and indirect data blocks, data blocks can contain different things. So, a data block can be classified into the following types depending on what data it contains:

1. "Real" data blocks (contains actual data of a file)
2. Directory data blocks (pointed to by the root directory inode, contains directory entries)
3. Indirect data blocks (contains an array of data block numbers)
4. Doubly-indirect data blocks (contain an array of indirect data block numbers)

Note: Since the root directory inode is treated similar to a "regular" inode, it can grow in size like other files with indirect and doubly-indirect data blocks. This means that the limiting factor on the number of files in a filesystem is typically the number of inodes.

7.2 File Abstractions

In order to make your filesystem driver, you should create a `struct ktfs_file`. This will be the internal representation of a file that your filesystem driver uses to keep track of the state of each file. This structure should contain *at least* the following fields (add more if you find it useful):

1. The `uio` struct associated with each file. The `intf` field in this structure should be a pointer to an `uio_intf` struct that contains the correct `close`, `read`, `write`, `cntl` function pointers to interface with the file.
2. A **file size** member that stores the length of the file in bytes. This should be updated whenever `write` is called, if the length of the file has changed.
3. The **directory entry** struct for this file. As a reminder, this contains the inode number for the file and the file name.
4. A **pos** argument that stores the current position in bytes for the file. This will need to be used for calls to `ktfs_cntl`, when `cmd` is `FNCTL_GETPOS` or `FNCTL_SETPOS`.

You need to have some way to keep track of the `ktfs_file` structs that correspond to currently opened files (array, linked list, etc.), so that future UIO operations can be performed on them.

7.3 Building the File System

The filesystem image needs to be made in order for QEMU to read it and make it accessible to you (via `VIRTIO`). To do this, we have provided you with a binary executable within the `util` directory.

`mkfs_ktfs` is a customized version of Linux `mkfs` that generates a filesystem image according to the KTFS spec detailed above. This is how we will generate filesystem images to test and grade your filesystem driver.

We will also provided you with another binary executable, `unmkfs_ktfs`. This program “unwraps” the files contained with a KTFS filesystem image to allow you to parse them on your own. You may find this useful when implementing the `write/writeat` functions in your filesystem driver, since these functions will modify your actual filesystem image and persist between QEMU shutdowns.

`mkfs_ktfs` usage is as follows:

```
./mkfs_ktfs [filesystem_image]
            [disk_size in K, M, or G]
            [inode_count]
            [file1] [file2] ...
```

In order for your filesystem image to work with our existing `Makefile`, you should name the image `ktfs.raw` and put it in the `sys` directory.

An example:

```
./mkfs_ktfs ../../sys/ktfs.raw 8M 16 ../../usr/bin/hello ../../usr/games/trek
```

This would create a `ktfs.raw` file in the `sys` directory with a total size of 8MiB and 16 inodes (*i.e.*, 1 inode block). `hello` and `trek` would be the only 2 files in the filesystem image, with the rest of the inodes free.

Since we cannot have fractional data blocks, your disk size should always be a multiple of 512 (KTFS block size). The number of inode blocks is also rounded up, since we cannot have fractional inode blocks. When

interfacing with the filesystem, you can consider any block marked as an “inode block” as able to be used to store inodes. In other words, the total number of possible inodes in your file system will always be a multiple of 16. However, bitmap blocks may be “partially” used. Be careful to not read/write past the size of the filesystem image.

Note: Make sure you are running your `mkfs_ktfs` binary and not the Linux `mkfs` function.

The files in your filesystem image are also given as arguments to `mkfs_ktfs`. You can (and should) provide multiple files in order to create a full filesystem image. All the metadata related to these files will be prepared for you, including the superblock and other structures that you’ve read about in **Appendix A** already. All of these blocks will be put in order according to the spec above and stored in your filesystem image (*i.e.*, `ktfs.raw`). This means that the filesystem image is already compliant with the KTFS spec.

To load a user program into the filesystem image, you need to compile it to an executable binary. This can be done by using the `Makefile` in `usr`. If you want to change a file in the filesystem or add/remove files (without using your filesystem driver), you **must** remake the filesystem image each time.

7.4 The Blob

The blob will be created as an object file and is not part of the regular filesystem. It is a tool for debugging and testing. If `blob.raw` exists in your `sys` directory, the `Makefile` will input the file contents of `blob.raw` to a read-only data section starting from `_king_blob_start` to `_king_blob_end`. These addresses can be obtained via `extern`. If you want to use the blob, simply rename the relevant file to `blob.raw` in the `sys` directory and add the object file when you build your kernel image ELF file as well as `extern` both addresses in your code to access the blob.

8 Appendix B: Uniform I/O

8.1 Overview

Due to the wide variety of different types of objects in our kernel, it can be very helpful to unify these under a common interface. You have already begun to see this with `serial` devices, which unifies many different types of devices into a single type of struct. Now, we will go even further, unifying all “file” like objects under a single Uniform I/O interface. We define file-like objects to be any object that can be read from and/or written to, and can be “closed”. This primarily applies to devices and files, but will go on to include listings, pipes, and generally any other form I/O.

8.2 Uniform I/O Operations

The kernel will interact with Uniform I/O using the struct `uio`. Each struct `uio` contains a pointer to a struct `uio_intf`, which in turn contains pointers to `close`, `read`, `write`, and `cntl`, functions specific to each endpoint. Note that not all these functions need to be implemented - if the device driver does not implement the function, the pointer can be `NULL`. Under the hood, these callback functions are endpoint specific and performs the relevant operation.

The `cntl` function allows control over endpoint specific operations that don’t map nicely into `read` and `write`. For example, objects with an internal position, such as storage devices and files, need some way for users to determine where to read from and write to. Thus these endpoint’s `cntl` functions accept `getpos` and `setpos` operations, defined as `FCNTL_GETPOS` and `FCNTL_SETPOS` in `uio.h`. Other endpoint specific operations are defined there as well.

8.3 Opening Uniform I/O objects

For object with similar endpoints, we group them together in the kernel via “mountpoints”. In our kernel, we will have the `dev` mountpoint (`devfs`), which will contain all devices, and the `c` mountpoint (`ktfs`), which will contain all files. We have a higher level wrapper around these mountpoints within `filesys.c`, which is responsible for mapping mountpoint names to their respective filesystems. Therefore, to open any file or device, you simply call `open_file` with the filename and its respective mountpoint name. When we open a file like this, we first iterate through our list of mountpoints to find the matching name, then call the open function attached to that filesystem. The filesystem itself is then responsible for finding the filename, opening it, and returning the `uioptr` with that file as its endpoint.

Note that for devices, the instance number is part of the filename. So, to open a UART device with an instance number of 1, you would call `open_file` on mountpoint name `dev` and file name `uart1`. You will have to do this in CP1 in order to pass in the UART device to `trek`, which will take in a `uio` object as an argument.

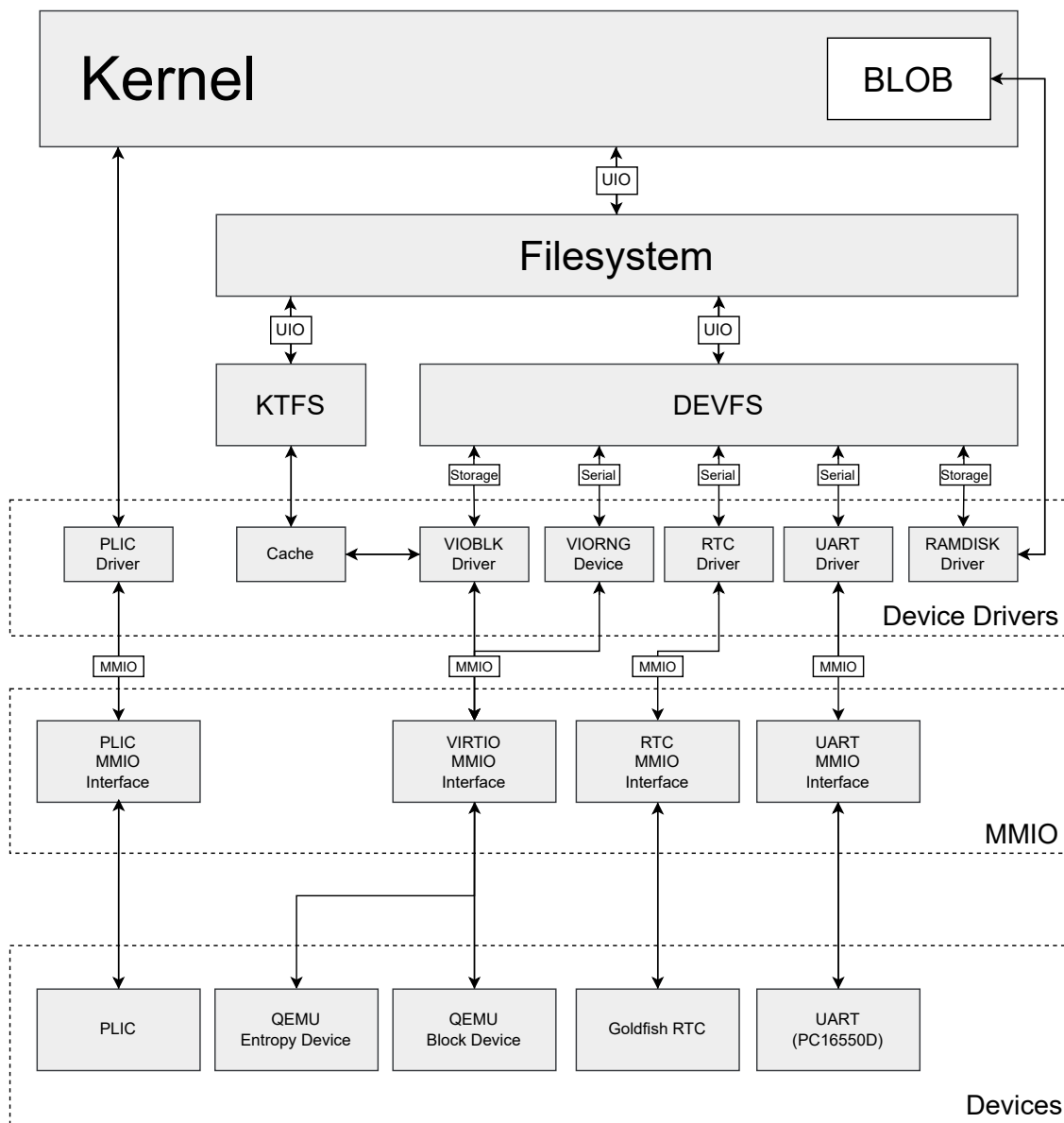
Please read through `filesys.h/c` and the `devfs` functions in `device.c` to gain a better understanding of how the system works.

8.4 Listing Objects

Listing objects are a special type of `uio` that are specific to mountpoints. It captures all files that are in the mountpoint when the listing object was opened. When passed in a `NULL` or empty string as a filename, this listing object of the mountpoint is returned instead. Calling `read` on the listing object will iterate through the filesystem files, returning a new filename on each call. This is what allows us to pass filesystem information to the user and eventually implement programs like `ls`.

There are three listing objects in your kernel. One is for `devfs` and is provided in release. Another is for `ktfs`, which you will have to implement in CP3. The final one is for `filesys.c`, which is returned when `open` is called and the mountpoint name is `NULL` or the empty string. This listing captures the list of mountpoints

8.5 Uniform I/O Diagram



Overall structure of the 391 Kernel.

9 Appendix C: The Process Abstraction

9.1 History

The process abstraction in Unix like systems is a critical part of understanding an operating system's inner workings. Historically, processes were the first abstraction over a user program rather than threads. User programs were seen as having a single flow of instruction and resided in separate memory spaces. As computer hardware advanced and provided stronger computing, it became desirable to have multiple streams of instructions (threads) running within a single address space. For Checkpoint 2 we will only consider processes that contain only a single thread.

9.2 Overview

The Linux kernel's smallest abstraction over a stream of instructions is a thread. For a UNIX-like system (like ours), each process by default will be spawned around a main thread. This main thread is what is scheduled by the operating system.

A process being scheduled by the kernel to run is just the process's thread being put on the ready list and a process running is just the thread associated with the process running (as the current thread).

On top of owning a thread, each process also owns a virtual memory space. This virtual memory space is represented by a 3-level page table. You can read more about the paging and virtual memory in **Appendix D**.

In addition, a process should also keep track of files and devices that it's interacting with. Note that we have a common interface for files and devices (`io`)

The process abstraction provides 3 key guarantees to a user process:

- **Protection:** Assures that a malicious process cannot corrupt, fault or deny other user processes.
- **Virtualization:** Creates the illusion to a process that resources are boundless and only used by that process, *i.e.* virtual memory, illusion of concurrency with scheduling, etc.
- **Abstraction:** Abstracts away implementation details and provides a common interface to user processes, *e.g.* a user program opening a file is the same code regardless of the file system being backed by a ramdisk or a vioblk device. They are covered by the same system call.

9.3 Context Switching between User-Mode and Supervisor-Mode

Context switching between two privilege levels fundamentally requires two instructions with two unique properties:

1. An instruction to go from a lower privilege level to a higher privilege level
2. An instruction to go from a higher privilege level to a lower privilege level

The ECALL Instruction:

The environment call (`ecall`) instruction will be our way of going from a lower privilege level (user-mode) to a higher privilege level (supervisor-mode). It is meant to be used by user-mode software to make a service request to a higher privilege context.

Reading the description for the `ecall` instruction in the RISC-V Privileged ISA manual we can see that it will trigger an exception which should be delegated to the supervisor-mode trap handler. This exception being triggered from user-mode will have the following effects:

1. The `sepc` register is loaded with the virtual address of the instruction that triggered the exception (*i.e.* the `ecall` instruction itself).
2. The `scause` register contains the cause code for the exception (the cause code for an environment call from user-mode).
3. The `sstatus` register's Supervisor Previous Privilege bits (SPP) will be set to 0 to indicate that it is a trap from user-mode.
4. The `sstatus` register's Supervisor Previous Interrupt Enable bits (SPIE) will indicate whether or not supervisor-mode interrupts were enabled prior to trapping into supervisor-mode.
5. The `sstatus` register's Supervisor Interrupt Enable bits (SIE) will be cleared to disable supervisor-mode interrupts while in supervisor-mode.

The SRET Instruction:

The supervisor return (`sret`) instruction is what we'll use to transition from a higher-privilege level (supervisor-mode) to a lower privilege level (user-mode). It is meant to return to user-mode once a user request has been completed.

Reading the description for the `sret` instruction in the RISC-V Privileged ISA manual we can see that an `sret` in supervisor-mode will return to a lower privileged mode (user-mode) with the following effects:

1. The `sstatus` register's SIE bit will be set to SPIE.
2. The `sstatus` register's SPIE bit will be set to 1.
3. The `sstatus` register's SPP bits will determine the execution privilege mode after `sret`.
4. The `sstatus` register's SPP bits will be set to the least-privileged supported mode (user-mode).
5. The `pc` register is set with the value of `sepc`.

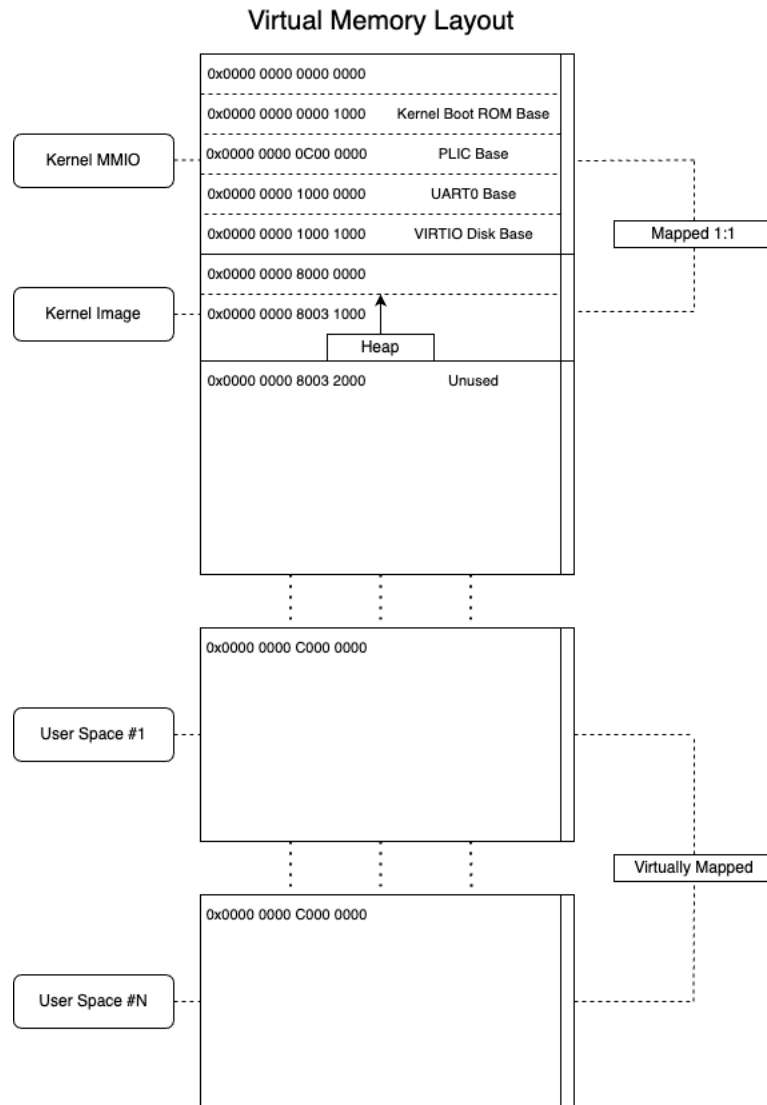
Now that we know the effects of `ecall` and `sret`, we can reason about how to use these instructions during context switches. There are two specific cases to consider:

- **Switching to supervisor-mode from user-mode:** This type of switch is the most straightforward transition and does not require any preparation. You can just use the `ecall` instruction.

- **Switching to user-mode from supervisor-mode:** Since our kernel initially boots into supervisor-mode, it can be a bit tricky to figure out how to use `sret` to transition into user-mode. This confusion may come up because at first glance `sret` requires a prior user program to have trapped into supervisor-mode (so that the SPP and SPIE bits will be pre-populated with values). However, if this is the first time a user program is being executed there can't be any prior user-mode trap frame and SPP and SPIE cannot be filled in. It is up to the `process_exec` function to fill in the proper values for SPP and SPIE so that `sret` can properly jump to a user-space function.

10 Appendix D: RISC-V Paging

10.1 Illinix Mappings



10.2 Sv39

RISC-V offers a few different paging schemes, each of which offers a different virtual memory size. We are using Sv39 for this MP. Each `struct pte_t` (defined in `memory.c`) describes a page table entry specific to Sv39. You will need to look at the RISC-V privileged docs to learn more about each field: [here](#).

Virtual memory address translation, also called **paging**, is a hardware-level tool that supports process virtualization. When paging is enabled and the machine is in `umode`, the hardware will translate all user-program memory accesses from virtual to physical addresses.

Multiple programs may be staged to execute at the same address. Memory virtualization allows each process to execute with the idea that they have the entire memory space at their disposal, and ensures that the operating system can manage many programs executing in parallel.

To use the QEMU console (useful in debugging) you can add the following line in `src/kern/Makefile` where you set your `QEMUOPTS`

```
QEMUOPTS += -monitor pty
```

When running `make run`, you will see another `/dev/pts/N` that you will need to connect to in order to interact with your QEMU console. To view the virtual memory mappings of your system, you can run `info mem` in your qemu console.

11 Appendix E: Troubleshooting

11.1 Debugging with gdb

“If debugging is the process of removing bugs, then programming must be the process of putting them in.”
-Edsger W. Dijkstra

One of the most important skills you can have as a software engineer is being able to debug. While print statements are useful and easy to implement, using a real debugger like gdb will allow you to find and solve your problems much more quickly. If you invest time into learning it at the start of this MP, it will pay dividends for the rest of the assignment and future classes. Many ECE 391 alumni (including course staff) have said that the most important thing they learned from the course was not how to make an operating system, but how to debug effectively on their own. While course staff are there to help you, they are not there to handhold you and it is expected that you are able to debug on your own, especially with gdb.

11.1.1 Debugging the Kernel

This section describes how to set up your kernel to work with gdb. This will allow you to step through kernel and user program code if it is compiled with debugging symbols (`-g`).

Whenever a change is made to your kernel, you should rebuild your kernel using `make`. For the purposes of this appendix, we’ll be describing debugging with the `kernel.elf` kernel image, which we’ve given you. If you’re using a different kernel image, use that instead. You can create different kernel images by writing an appropriate `main_<prog>.c` file and adding it to the `Makefile`.

To launch QEMU and have it wait for remote debugging, you must include the `-S` flag, which we have done for the command `make debug`. This will launch your `kernel.elf` kernel image as normal, but no instructions will execute until you connect to it with gdb and run `continue`.

Once you’ve launched your kernel, you need to open a second terminal and run

```
riscv64-unknown-elf-gdb [kernel image]
```

This will run the correct version of gdb and load the debugging symbols from your kernel image, allowing you to set breakpoints and step through the code. We have included a `.gdbinit` file in the `sys` directory, which executes a few commands on gdb startup — this should cause it to automatically connect to your QEMU instance. To use the `.gdbinit` file, simply launch gdb from the `sys` directory.

Note: The first time you use a `.gdbinit` file, you might get a message that looks like this

```
warning: File "/path/to/src/kern/.gdbinit" auto-loading has been declined by
your 'auto-load safe-path'.
```

To enable execution of this file add

```
add-auto-load-safe-path /path/to/src/kern/.gdbinit
```

line to your configuration file `"~/.config/gdb/gdbinit"`.

In order to resolve this, you should simply follow the instructions that gdb gives you and add the path. You may have to use `mkdir` to make the `~/.config/gdb` directory before using your favorite text editor to add

the `add-auto-load-safe-path /path/to/src/kern/.gdbinit` line into `~/.config/gdb/gdbinit`. You only need to perform this setup process once per `.gdbinit` file, even if you modify it in the future. It is **highly recommended** that you use the `.gdbinit` file to save you the trouble of having to connect to the QEMU instance every time.

Note: You may need to modify the `target remote 127.0.0.1:1234` line in your `.gdbinit` file to match the port your QEMU instance uses. The `-s` flag provided to QEMU allows it to use the default port (1234), but if you are having trouble connecting with that port, you can explicitly specify which port to use in the QEMU command. Then, modify your `.gdbinit` file to match.

11.1.2 Debugging User Programs

If you want to set breakpoints or step through a user program, things are slightly more complicated. Since `kernel.elf` (or whatever your kernel image is) does not contain the user program, we need to add the debugging symbols for it separately into `gdb`.

First, you need to compile your user program with debugging symbols enabled. In the `Makefile` we have provided, we have already enabled debugging symbols (`-ggdb3`). If you write your own user program, you can add it to the `Makefile`. To find the address of the program itself, run

```
readelf -WL [program binary]
```

This gives you information on where different sections of your program are loaded. Find the executable section (E flag) and note down the `VirtAddr`. For checkpoint 1, it should be located around `0x80100000`. For later checkpoints, once you have virtual memory enabled, it should be located between `USER_START_VMA` and `USER_END_VMA`. If using `UMODE` and virtual memory, be sure to enable the `UMODE` flag within `usr/Makefile`. You may need to manually tell `gdb` that this is where the program starts.

To do this, open `gdb` as normal with `make debug`, then use the `add-symbol-file` command to add your user program. An example would be

```
add-symbol-file ../usr/bin/hello
```

If there is a prompt, hit “Y” to accept. Now, you should be able to set breakpoints and step through the user program. You will need to run this command every time that you re-open `gdb` — if you find yourself using it constantly, consider adding it to your `.gdbinit`.

11.1.3 Useful gdb Commands

The best way to get good at using `gdb` is to actually use it. To get you started, here are some pages with `gdb` commands we found useful. There are a lot more out there, and it is highly encouraged that you spend time looking at the `gdb` man page (`man gdb`) and any other online resources for `gdb`.

Note: Your code may run noticeably slower when using `gdb`, especially if you are performing a lot of comparisons.

- Stanford CCRMA `gdb` Guide
- `gdb` Wiki

Here are some additional useful gdb tips and tricks:

1. Manipulating Local Variables

You can use arithmetic operations (+, -, *, /, etc.) as well as pointer arithmetic (*, &, [], etc.) on variables in gdb. You can also cast variables or numbers, just like in C. To access the fields of a struct, use `.` or `->` appropriately. This is most useful with `p[rint]`.

2. Watchpoints

You can use watchpoints to “watch” the value of a variable and trigger a breakpoint if it changes. Use `watch <var>` to set a watchpoint on a variable.

3. Conditional Breakpoints and Watchpoints

You can use the format `break WHERE if CONDITION` to set a conditional breakpoint, where it will only happen if a certain condition is met. For example, `watch i if i == 100` would break if `i == 100`. If your variable is constantly being modified, this can cause a major slowdown in your program execution.

4. Printing Registers

Using `i[nfo] r[egisters]` can be useful to see all (or any specific) registers, but all registers can also be referenced with the `$<register name>` variable. For example, `p/x $sp` prints out the value of `sp` in hexadecimal. However, for CSRs, it is recommended you use `i[nfo] r[egisters]` in order to get “pretty printing”.

5. gdb History

By adding `set history save on` to your `.gdbinit` file, you can access the history of commands from previous gdb sessions. You can also further customize the file location and history depth.

6. `until` and `finish`

When using gdb, stepping through long for loops can be somewhat annoying. Typically, you will set a breakpoint after the loop and use `c[ontinue]` to skip over it. However, you can also use the `until <line number>` command to skip without having to set a breakpoint.

`fin[ish]` works in a similar way. It runs until the current function exits, then reports the return value of the function. You may find this useful if you accidentally `s[tep]` instead of `n[ext]`.

12 Appendix F: Games

12.1 Overview

In the release, we have provided 3 games in under the `games/` directory: Trek, Rogue, and Zork. Here are some notes for anyone interested in playing these games, *NOT FOR TESTING OR DEBUGGING YOUR OS!* All executables expect the UART they are outputting to to be opened at descriptor 2 in their process's Uniform I/O table, and additionally will attempt to open the RTC (“rtc”) and Viorng (“viorng”).

12.2 Trek

Trek is just the same as its been throughout the semester, hopefully you have picked up the basics of Trek by now.

We have also provided two other classic text based computer games: Zork and Rogue. Both programs contain functionality to save and restore data, allowing you to maintain your progress just like in a modern game! Assuming your filesystem works, that is. . .

12.3 Zork

Zork takes no arguments, and saves progress to a file called `dsave.dat`. Accordingly, a file called “`dsave.dat`” must be added to your filesystem to properly save and restore data for Zork. If it does not exist, Zork will attempt to create it using your `_fscreate` syscall in your `c` drive. All game data for Zork is stored in the `dtextc.dat` file which is included in the distribution for the program. Game data is saved by typing “save” into the zork terminal, and saved game data is restored by typing “restore” into the zork terminal. The manual for zork is available online for other commands, and general information about the game.

Zork manual: <https://infodoc.plover.net/manuals/zork1.pdf>

12.4 Rogue

Rogue optionally takes in the name of the file to restore saved data from. Note the program is written like a typical command line executable with `argc` and `argv`, and the first argument is expected to be the name of the executable. Thus, the name of the file to restore saved data from must be the second string in `argv`. Data can be saved at any time by entering `S` (capitalized), and entering the name of the file that data is to be saved to. This will attempt to write to the specified file, or create it if it does not exist. Remember to include the full path! Note saving will cause the program to exit after saving your data.

The manual for Rogue is also available online, and should prove quite useful since many of the commands are unintuitive (q to quaff a potion????). You may also enter ‘?’ into the rogue screen to get info on commands and objects in the dungeon. Also note that you must press space when the output message contains “-more-” to read the next message and continue with the game. Good luck in your quest to find the sacred Amulet of Yendor!

Rogue manual: <https://britzl.github.io/roguearchive/files/misc/EpyxRogueDOSManual/manual.htm>

12.5 Game Quirks

- For rogue to be printed to terminal correctly, there must be at least 24 rows on your terminal.
- Zork and Rogue expect file descriptors 3 on to be free to work correctly (they will attempt to open rtc, viornrg, and finally any needed files starting from FD 3). Both work without rtc and viornrg working, but zork will not work if it cannot open dtextc.dat and read game data.

Good luck and have fun! Praise Kier!